

Advanced Programming Java

Sessional I - Detailed Solutions

Q-1. Explain the importance and features of Java. And also explain Java Virtual Machine (JVM).

Importance and Features of Java:

Java is one of the most widely used programming languages, particularly for enterprise applications, Android app development, and large-scale systems. Its importance lies in its core features:

- **Platform Independent (Write Once, Run Anywhere):** This is Java's most significant feature. Java compiler converts source code into bytecode, which is platform-agnostic. Any system with a JVM can run this bytecode.
- **Object-Oriented:** Everything in Java is an object (except primitive data types). It models real-world entities using classes and objects, supporting concepts like inheritance, polymorphism, encapsulation, and abstraction.
- **Simple and Familiar:** Java was designed to be easy to learn. Its syntax is based on C++, but it removes complex features like explicit pointers and operator overloading.
- **Robust:** Java emphasizes early checking for possible errors. Features like strong memory management (Garbage Collection) and exception handling make it highly reliable.
- **Secure:** Java has no explicit pointers, and programs run inside a virtual machine sandbox. It also includes built-in security features for authenticating components.
- **Multithreaded:** Java has built-in support for multithreading, allowing a program to perform multiple tasks simultaneously, which is essential for multimedia and high-performance applications.

Java Virtual Machine (JVM):

The JVM is an abstract computing machine that enables a computer to run a Java program. It is the engine that drives the Java code.

- **Role:** When you compile a `.java` file, the Java compiler produces a `.class` file containing bytecode. The JVM translates this bytecode into machine-specific code that the underlying operating system can execute.
- **Key Components:**
 1. **Class Loader:** Loads the compiled bytecode into memory.

2. **Memory Area:** Allocates memory for the program (Method Area, Heap, Stack, PC Registers, Native Method Stacks).
3. **Execution Engine:** Contains the interpreter (reads bytecode line by line) and the JIT (Just-In-Time) compiler (compiles frequently used bytecode to native machine code for faster execution).

Q-2. Define the following: 1. Data type 2. Keywords 3. Constants 4. Variables

1. Data Type:

A data type specifies the size and type of values that can be stored in an identifier (variable). It tells the compiler how much memory to allocate and what kind of operations can be performed on the data. In Java, data types are broadly categorized into Primitive (e.g., `int`, `char`, `boolean`) and Non-Primitive (e.g., `String`, `Array`, `Class`).

2. Keywords:

Keywords (or reserved words) are predefined words in Java that have a specific, intrinsic meaning to the compiler. They cannot be used as variable names, class names, or method names. Examples include `class`, `public`, `static`, `void`, `if`, `for`, and `int`.

3. Constants:

A constant is a variable whose value cannot be changed once it has been assigned. In Java, constants are created using the `final` keyword. By convention, constant names are written in ALL_CAPS.

- Example: `final double PI = 3.14159;`

4. Variables:

A variable is a named container (memory location) used to hold data while a Java program is executed. Every variable must be declared with a specific data type before it can be used.

- Example: `int age = 20;` (Here, `age` is the variable).

Q-3. Explain control and conditional statements in java with syntax.

Control and conditional statements determine the flow of execution in a Java program based on certain conditions or rules.

1. Conditional (Decision-Making) Statements:

These evaluate a boolean expression and execute code based on whether the result is `true` or `false`.

- **if statement:** Executes a block of code if a condition is true.

```
if (condition) {  
    // block of code to be executed if condition is true  
}
```

- **if-else statement:** Executes one block if true, and another if false.

```
if (condition) {  
    // block of code if condition is true  
} else {  
    // block of code if condition is false  
}
```

- **else-if ladder:** Used to check multiple conditions in sequence.

```
if (condition1) {  
    // code if condition1 is true  
} else if (condition2) {  
    // code if condition2 is true  
} else {  
    // code if all conditions are false  
}
```

- **switch statement:** Selects one of many code blocks to be executed based on a variable's value.

```
switch(expression) {  
    case x:  
        // code block  
        break;  
    case y:  
        // code block  
        break;  
    default:  
        // default code block  
}
```

2. Control (Looping) Statements:

These execute a block of code repeatedly as long as a condition is true.

- **for loop:** Used when the number of iterations is known.

```
for (initialization; condition; increment/decrement) {  
    // code block to be executed  
}
```

- **while loop:** Used when the number of iterations is not known beforehand.

```
while (condition) {  
    // code block to be executed  
}
```

- **do-while loop:** Similar to `while`, but guarantees the code block executes at least once before checking the condition.

```
do {  
    // code block to be executed  
} while (condition);
```

Q-4. What is string? Explain String Buffer, StringBuilder classes and String methods.

What is a String?

In Java, a String is an object that represents a sequence of characters. It is backed by the `java.lang.String` class. The most important characteristic of a Java String is that it is **immutable**, meaning once a String object is created, its value cannot be changed. Any operation that seems to modify a String actually creates a new String object.

StringBuffer Class:

`StringBuffer` is a peer class of `String` that provides much of the functionality of strings but is **mutable** (modifiable).

- **Features:** It is thread-safe (synchronized), meaning multiple threads cannot access it simultaneously. This makes it safer for multithreaded environments but slightly slower.

StringBuilder Class:

`StringBuilder` is also used to create mutable strings. It was introduced in Java 1.5 as a drop-in replacement for `StringBuffer`.

- **Features:** It is **not** thread-safe (non-synchronized). Because it doesn't have the overhead of synchronization, it is significantly faster than `StringBuffer` and is preferred in single-threaded environments.

Common String Methods:

- `length()` : Returns the number of characters in the string.
- `charAt(int index)` : Returns the character at the specified index.
- `substring(int beginIndex, int endIndex)` : Extracts a portion of the string.
- `toLowerCase()` / `toUpperCase()` : Converts the string to lower or upper case.
- `equals(Object another)` : Compares the content of two strings for equality.
- `concat(String str)` : Appends a string to the end of another.

Q-5. Describe data types in Java and explain type casting with examples.

Data Types in Java:

Java is a strongly typed language. Data types are split into two categories:

1. **Primitive Data Types (8 types):** Built-in types that hold pure values.
 - *Integer types:* `byte` (1 byte), `short` (2 bytes), `int` (4 bytes), `long` (8 bytes).
 - *Floating-point types:* `float` (4 bytes), `double` (8 bytes).
 - *Character type:* `char` (2 bytes, uses Unicode).
 - *Boolean type:* `boolean` (1 bit, holds `true` or `false`).
2. **Non-Primitive (Reference) Data Types:** These refer to objects and include Classes, Interfaces, Arrays, and Strings.

Type Casting in Java:

Type casting is the process of converting a value from one primitive data type to another. There are two types:

1. **Widening Casting (Implicit Casting):**

Converting a smaller data type to a larger data type size. This is done automatically by Java because there is no risk of data loss.

◦ *Hierarchy*: `byte` → `short` → `char` → `int` → `long` → `float` → `double`

```
int myInt = 9;
double myDouble = myInt; // Automatic casting: int to double
// myDouble is now 9.0
```

2. **Narrowing Casting (Explicit Casting):**

Converting a larger data type to a smaller size. This must be done manually by placing the target type in parentheses in front of the value, as there is a risk of losing precision or data.

◦ *Hierarchy*: `double` → `float` → `long` → `int` → `char` → `short` → `byte`

```
double myDouble = 9.78;
int myInt = (int) myDouble; // Manual casting: double to int
// myInt is now 9 (the decimal is truncated)
```